

AI Course

Capstone Project Final Report

For students (instructor review required)

© 2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of this document.

This document is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.

To use this document other than the curriculum of Samsung Innovation Campus, you must receive written consent from copyright holder.

Bayan: On-Device Arabic Text Simplification for Readers with Dyslexia

تبسيط · Simplify

28/09/26

Cogni

Marwan Elamami
Abdulrahman Khengari
Ahmed Alaeb
Sanad Ali
Mohammed Thabet
Abdul Majid Mraied

Content

1. Introduction

- 1.1. Background Information
- 1.2. Motivation and Objective
- 1.3. Related Work
- 1.4. Members and Role Assignments
- 1.5. Schedule and Milestones

2. Project Execution

- 2.1. Data Acquisition
- 2.2. Training Methodology
- 2.3. Workflow
- 2.4. System Design
- 2.5. BayanBench: a Benchmark for the Reader's Task

3. Results

- 3.1. Data Preprocessing
- 3.2. Exploratory Data Analysis (EDA)
- 3.3. Modeling
- 3.4. On the Device
- 3.5. User Interface
- 3.6. Testing and Improvements

4. Projected Impact

- 4.1. Accomplishments and Benefits
 - 4.2. Future Improvements
 - 4.3. Limitations
 - 4.4. Ethical Considerations
- References

5. Team Member Review and Comment

6. Instructor Review and Comment

Abstract. Arabic poses readers with dyslexia problems that tools built for Latin scripts do not address: short vowels are left unwritten, clitics fuse into long words, and clauses chain across whole paragraphs. We present **Bayan**, an Android system that rewrites a selection of Arabic text into a form that is easier to decode, from inside any application and entirely on the phone. We make four contributions. **(1) Data.** We audited the public Arabic simplification corpora before training on them and found that 9.3% of one of them overlaps our locked test sets. We then built a 22,733-pair training set with every unchanged pair kept, LLM-written pairs admitted only through a meaning filter, and a strength tag on every source. **(2) Models.** We fine-tuned two encoder–decoder transformers, AraT5v2 (368M parameters) and AraBART (139M), and show why a plain SARI score misleads: the copy baseline scores 77.5 on SAMER test, so our first model needed a separate score on the sentences a human actually changed. **(3) BayanBench.** We built a behavioural benchmark of 1,981 items in 13 tracks and four item sets, split by document. It reports each measure separately with a cluster-bootstrap interval, never as a composite, and separates deterministic code checks from LLM-judge verdicts. **(4) On the device.** The int8 AraBART bundle (222 MB) runs inside the app with streamed output and read-aloud; on a Snapdragon 870 phone, the first token of a sentence arrives in 55–318 ms. On the BayanBench test split, model 2 removes every clause over 15 words in 50.8–55.1% of long-clause items and reduces hard words in 41.2–44.0%. It also exposes failures a SARI score hides: it edits *easy* text that should be left alone, rewrites protected scripture, and drops some Persian-script digits. Each is traced to a concrete fix.

1. Introduction

1.1. Background Information

Developmental dyslexia affects an estimated 11% of Arab primary-school children, pooled across 18 studies ($N = 30,243$) [1]. It impairs reading through phonological processing, and Modern Standard Arabic adds obstacles that tools built for Latin scripts do not address. Short vowels are normally left unwritten, so one written form carries several readings: **كتب** is *kataba* (he wrote), *kutiba* (it was written) or *kutub* (books), and the reader must resolve it from context. Clitics attach directly to the stem, producing long words that resist chunking, and readers with reading disabilities show early deficits in exactly this morphological layer [2]. Prose chains clauses with **و** and **ف** across whole paragraphs, so a single sentence can exceed working memory.

The evidence on what helps is uneven, and we built on the parts that are measured. More frequent and shorter words sped reading and improved comprehension for readers with dyslexia in eye-tracking studies [3, 4]. Unpacking bound morphemes into separate words helped Arabic readers with reading disabilities and not their typical peers, the one Arabic finding with a tested group interaction [5]. Texts simplified by rules that split subordinate and coordinated clauses into short sentences were read faster and understood better by beginning readers, and most by the weakest of them [6]. Full diacritization, by contrast, is contested: it adds visual crowding [7], and its benefit for our readers is untested. We therefore treat *simplify first* as the core intervention and diacritization as an optional later stage.

When the project started we found no deployed Arabic dyslexia-support tool that simplifies text, and no large, openly reusable Arabic simplification corpus. That shaped both what we built and how much of our effort went into data and evaluation.

1.2. Motivation and Objective

Readers meet difficult Arabic inside other applications — a message, a news page, a PDF — not in a reading app. Bayan therefore appears where the text already is: selecting text on an Android phone offers تبسيط (*Simplify*) in the system text-selection menu, and the simplified text opens in a panel over the same app. Every model runs on the phone, so the reader’s text never leaves it and the tool works offline.

We committed to four measurable objectives; Table 18 in subsection 4.1 reports where each one stands on 28 September.

- 01** A simplifier that beats *copying the input* on the sentences a human actually simplified, on held-out test data it never saw, and that makes text measurably easier without losing its meaning.
- 02** Diacritics restored on the simplified text without altering its letters.
- 03** The whole pipeline running on a mid-range Android phone, within 250 MB for the simplifier.
- 04** Evidence from readers: a human evaluation with Arabic readers who have dyslexia, with informed consent.

Contributions. Beyond the product, the work contributes:

- **A contamination audit** of a public Arabic simplification corpus (9.3% of Baseet overlaps the SAMER and BAREC test sets), and a leakage gate that every training file must pass (subsection 3.1).
- **A training recipe** for one model with several strengths: all unchanged pairs kept, LLM-written pairs admitted only if they keep the meaning, and a strength tag that becomes the app’s setting (subsection 2.2).
- **BayanBench**, a behavioural benchmark for dyslexia-oriented Arabic rewriting, released as a Python package with a judge-acceptance protocol fixed before any candidate judge was tested (subsection 2.5).
- **An on-device system**: int8 ONNX encoder–decoder bundles decoded token by token inside an Android app, measured on a phone (subsection 3.4).

1.3. Related Work

Arabic simplification data. SAMER [8] is the only human-written multi-level Arabic simplification corpus: 15 novels rewritten at three levels, with the same number of sentences in every version by design. BAREC [9] labels 69,441 sentences with 19 human reading levels but is not parallel. Baseet [10] provides three LLM-written levels per source, and DAASI [11] human-verified rewrites of government and insurance text. No corpus targets readers with dyslexia, and none teaches sentence splitting: in our measurements a sentence is added in 0% of SAMER pairs, 2–4% of Baseet’s, and 8% of DAASI’s.

Controllable simplification. Prepending a control token to the source lets one model produce rewrites for different audiences, first with target-grade tokens [13] and then with measured attributes such as length and lexical complexity (ACCESS, [14]; MUSS, [15]). Our strength tags follow this line, with one difference: each tag names the corpus style it was learned from, so that a meaning filter can be applied per source.

Evaluating simplification. SARI [16] is the field’s standard, but it correlates only $r = 0.36$ with human simplicity judgements [17], and it is blind to meaning errors: similarity metrics detect deletions but not insertions or substitutions [18]. Reference-free level estimates track simplicity better [19], and edit-level error taxonomies track meaning [20]. For benchmark design we follow behavioural testing [21], construct-validity and reporting checklists [22, 23], and document-clustered intervals [24]. LLM judges carry a known risk of favouring outputs close to data they produced or approved [25], which shapes how we restrict ours (subsection 2.5).

1.4. Members and Role Assignments

Every member has merged work into the team repository; the evidence column cites the pull requests and issues that hold it (#).

Table 1. Members, roles, and the work each has delivered by 28 September.

Member	Role	Delivered (evidence)
Marwan Elamami	Team lead; integration; on-device model	Product-form decision (#9); evaluation protocol and fixes (#16, #18); model 1 diagnosis; Baseet acquisition and leakage filtering; model-2 recipe, data and task specifications (#27–#34); on-device engine and app rebuild (#32); BayanBench; Action Plan (#25); this report
Abdulrahman Khengari	Data lead	Validators for generated pairs (#10, #11); CAMEL readability check (#17, #20); synthetic corpus of 13,072 pairs, split and published (#8, #38, #39); review follow-ups (#42–#44)
Ahmed Alaeb	Model training	SAMER conversion; model 1 trained and documented (#5, #22); model 2 on AraT5v2 trained and published (#28, #40)
Sanad Ali	Diacritization; co-training	Four-system diacritization benchmark; Libtashkeel selected (#6, #23); model 2 on AraBART and the speed benchmark (#29, #35)
Mohammed Thabet	Android application	PROCESS_TEXT plugin, result sheet, 41 tests (#14, #21); model download pipeline, sheet states and typography settings (#31, PR #37)
Abdul Majid Mraied	Evaluation	Scoring and leakage scripts (#7, #15); locked-set converter, copy baselines, generate-and-score path (#19, #24, #26); model-2 test tables (#30, in progress)

1.5. Schedule and Milestones

The project runs from 13 September to the pitch on 6 October 2026. At the time of writing (28 September, day 15 of 23) two full build-and-measure cycles are complete: model 1 was trained, scored on locked test sets and diagnosed; model 2 was designed from that diagnosis, trained on two architectures, exported to the phone and scored on a benchmark built for the task. Figure 1 shows the work as executed; Table 2 compares the dates we set with the dates we met.

2. Project Execution

2.1. Data Acquisition

We acquired five corpora and produced a sixth (Table 3). SAMER is used under written approval from CAMEL Lab to fine-tune and to publish the resulting weights non-commercially; the corpus itself is never redistributed, and no SAMER sentence is reproduced in this report. Baseet is

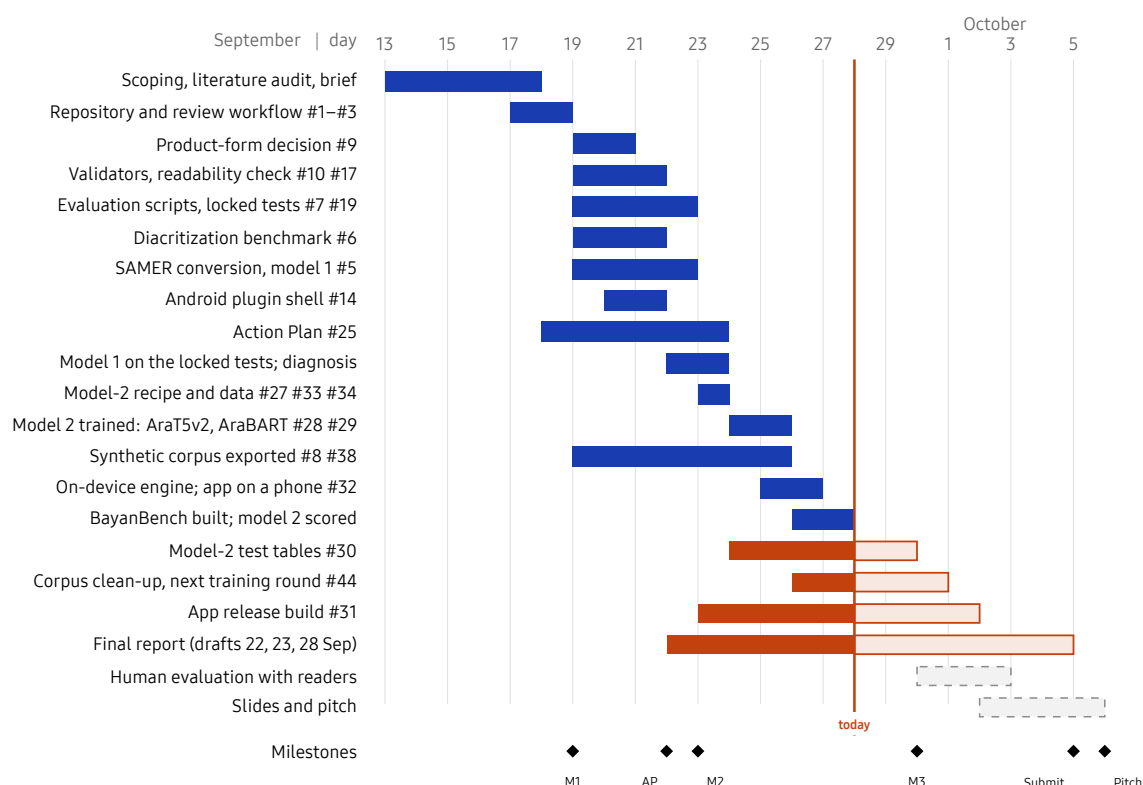


Figure 1. Project timeline as executed. Blue: done. Orange: in progress, solid to today. Dashed: not started. AP = Action Plan; M1–M3 = mentoring sessions. Numbers after a task are its GitHub issues.

used with its authors’ permission. Two test splits — SAMER test and BAREC test — were locked before any training and are never trained on.

The synthetic corpus. Abdulrahman’s pipeline takes BAREC train and dev sources, generates five candidates per source with DeepSeek V4.1 Flash, and accepts a candidate only if an LLM equivalence validator (a DSPy program optimised with MIPROv2, 84% accuracy on 25 held-out annotated pairs at the chosen threshold) and the CAMEL BAREC readability model both pass it. The export is split by a salted hash of the normalised source, so a sentence repeated under several BAREC IDs never straddles two splits, and a classifier trained to tell the splits apart scores AUC 0.51, indistinguishable from chance. BAREC test was never used. Code-only checks after publication (#44) found classes of pairs the app should not learn from — quiz items (11.5%), sources under five words (8.1%), quoted Qur’anic verses paraphrased, and cut-off sources completed with a full stop (22.4%). The corpus is therefore held back for the next training round, and model 2 was trained without it.

2.2. Training Methodology

Candidate base models. Simplification is a sequence-to-sequence task, so every candidate is a pretrained encoder–decoder transformer (Table 4). Architecture figures are read from each model’s configuration file; parameter counts from its checkpoint.

AraT5v2 went first because, on Baseet’s head-to-head evaluation (same split, same harness), it beat AraBART at all three simplification levels. AraBART is the on-device contender because it is 2.6 times smaller. HPLT T5 is the reserve.

Why the input is stripped of tashkeel. None of the three vocabularies was built for diacritized

Table 2. Milestones: target against actual.

Milestone	Target	Actual	Status
Evaluation scripts ready (#7)	Mon 21 Sep	Mon 21 Sep	Done
Diacritizer selected (#6)	Mon 21 Sep	Mon 21 Sep	Done
Android plugin shell (#14)	—	Mon 21 Sep	Done
Action Plan submitted	Tue 22 Sep	Tue 22 Sep	Done
Model 1 scored on the locked test sets	Wed 23 Sep	Wed 23 Sep	Done
Model-2 data built; tasks issued (#28–#32)	—	Thu 24 Sep	Done
Model 2 trained, both architectures (#28, #29)	Fri 25 Sep	Fri 25 Sep	Done
Synthetic corpus exported (#8, #38)	Fri 25 Sep	Sat 26 Sep	Done
Model running on a phone, size and latency measured	Fri 2 Oct	Sat 26 Sep	Done
BayanBench built; model 2 scored on its test split	—	Mon 28 Sep	Done
Model-2 test-set tables with SARI (#30)	Sat 26 Sep	—	In progress
Human evaluation with readers	Sat 3 Oct	—	Not started
Final submission / pitch	5 / 6 Oct	—	Not started

Table 3. Corpora. Sizes are rows in the files we hold.

Corpus	Size	Licence	Used for
SAMER [8]	14,343 / 2,983 / 3,277 pairs (train/dev/test)	research; written approval	models 1 and 2; locked test set
BAREC [9]	54,845 / 7,310 / 7,286 sentences	CC BY-SA 4.0	locked test set; BayanBench core items; sources for generation
Baseet [10]	34,294 sources, three levels each; 31,104 after filtering	with permission	model 2 (6,888 meaning-filtered pairs)
DAASI [11]	1,509 pairs + 350 held out	CC0	model 2 (1,502 pairs); out-of-domain test
Tashkeela [12]	743 test sentences	open	diacritization benchmark
Bayan synthetic v0 (#39)	13,072 pairs: 12,419 generated, 563 identity, 90 scripture	CC BY-SA 4.0, published	next training round, after clean-up (#44)

text: in AraT5v2 only 1.6% of pieces carry a diacritic and a diacritized sentence costs 2.57 times as many tokens; AraBART cannot represent a diacritic at all. subsection 3.3 measures what this does to model 1: on BAREC sentences that carry tashkeel, raw input makes the model drop more than half of the sentence in 31.9% of outputs. Every model therefore receives the source with tashkeel and tatweel removed, in training and in the app.

Model 1. We fine-tuned AraT5v2-base-1024 [26] to rewrite SAMER’s level-5 sentences into their level-3 simplifications: AdamW (learning rate 10^{-4} , weight decay 0.01, 100 warm-up steps), an effective batch of 16, mixed precision, 10 epochs (3,250 steps), seed 42, and the task prefix :بسط (simplify:). Pairs whose simplification is identical to the source (42% of SAMER train) were capped at 20% of training, and the checkpoint with the best SARI on a 500-row dev subset was kept.

Model 2: what model 1 taught us. Model 1’s diagnosis (subsection 3.3) found three problems, and the model-2 recipe answers each with one change:

- (a) *It rewards doing nothing, for the wrong reason.* Capping unchanged pairs at 20% and choosing the checkpoint on a dev sample that is 43% unchanged pairs pushes the model both

Table 4. Candidate base models. “Tashkeel” is measured on 1,000 diacritized BAREC dev sentences: vocabulary entries containing a diacritic, and how many more tokens the same sentences cost with diacritics than without.

	AraT5v2-base-1024	AraBART	HPLT T5 base (Arabic)
Family	T5 [29]: span-corruption pretraining, relative position buckets	BART [30]: denoising pretraining, learned positions	T5 in the NorT5 variant [31]
Layers (enc/dec), width	12/12, $d=768$, 12 heads, gated-GELU FFN 2048	6/6, $d=768$, 12 heads, GELU FFN 3072	12/12, $d=768$, 12 heads, FFN 2048
Parameters	368M (46% in two untied $110k \times 768$ embeddings)	139M	$\approx 294M$
Vocabulary	110,208 SentencePiece	50,002 SentencePiece BPE	32,768
Context	1,024 tokens	1,024 positions	512 positions
Pretraining	Arabic MSA and dialects; v2 of AraT5 [26, 27]	large Arabic corpora [28]	HPLT v3 web crawl, Arabic subset [32]
Tashkeel	1.6% of pieces; $2.57 \times$ tokens	0 pieces; every mark becomes <code><unk></code> ; $4.53 \times$ tokens	0 pieces; $1.81 \times$ tokens
Licence	none stated on the model card	Apache-2.0	Apache-2.0
Status	models 1 and 2	model 2	held in reserve

ways at once. Model 2 keeps *all* 6,033 unchanged SAMER pairs, so it learns when not to edit, and chooses its checkpoint on SARI over dev rows a human changed only.

- (b) *SAMER is novels, and it teaches word swaps.* Model 2 adds DAASI’s everyday and administrative text and Baseet’s educational text. Baseet’s targets were written by an LLM, and many are summaries: they drop facts, delete answer options, even invent a speaker. A Baseet pair is therefore kept only if it passes a *meaning filter*: LaBSE similarity ≥ 0.646 , target at least 67% of the source’s length, and every number kept. Each threshold is the 5th percentile of the same measure over DAASI’s human-verified rewrites, so a pair is rejected only when it changes the source more than 95% of human simplifications do.
- (c) *One strength does not fit every reader.* Every source carries a tag that names the style it was learned from (Table 5); at run time the tag is the app’s strength setting, so one model serves light, medium and strong rewrites [13, 14].

Table 5. Model-2 training set (22,733 pairs). Sources carry their tag; tashkeel and tatweel are removed. For each Baseet source the mildest level whose target is shorter per sentence or has fewer rare or long words is used. No source overlaps any test set.

Tag	Taught by	Pairs	Meaning at run time
[S0]	SAMER L5→L3, all pairs (8,310 changed, 6,033 unchanged)	14,343	minimal: literary word swaps
[S1]	Baseet level 3, meaning-filtered	3,508	light
[S2]	Baseet level 2, meaning-filtered	1,954	medium (the default we score)
[S3]	Baseet level 1, meaning-filtered	1,426	strong
[SA]	DAASI train	1,502	everyday and administrative style

Model 2: training. Both architectures were trained on exactly the same data with one shared script (`train.py`, #40), so that the architecture is the only difference: 10 epochs, batch 8 with 2 gradient-accumulation steps, fp16, maximum length 256, seed 42, evaluation twice per epoch

with patience 3, and the checkpoint chosen on SARI over the 600 changed rows of a fixed 900-row SAMER dev sample. AraT5v2 used learning rate 10^{-4} and the prefix: بَسط; AraBART used $5 \cdot 10^{-5}$ and : بَسط without the shadda, because its tokenizer maps the shadda to an unknown token. One guard is built into the script: in the pinned transformers 4.46, grouping batches by length also shuffles the evaluation set, which would score every prediction against another row's reference and pick the checkpoint at random. Grouping is off, and the script stops if evaluation rows and references ever fall out of alignment; the logs show 100% alignment at all 40 evaluations.

How it is evaluated. Two layers. Against published work and human references, every SARI [16] is reported beside the copy baseline and split into rows a human changed and rows a human left unchanged. For the reader's actual task, BayanBench (subsection 2.5) reports behaviour track by track. No training or selection decision used a test score.

2.3. Workflow

Work runs through GitHub: each task is an issue with an owner, a reviewer and a definition of done; changes arrive as pull requests, are reviewed, and are squash-merged. By 28 September the team had opened 20 issues (14 closed) and 24 pull requests (20 merged), with work from every member. Private data (SAMER text, model outputs on SAMER) moves only through private Kaggle datasets and a private Hugging Face organisation; the repository is kept free of it by rule and by .gitignore.

Figure 2 shows the data path. Three gates can reject data. The leakage check compares every training file against both locked test sets, by exact match and by character 5-gram Jaccard overlap of 0.8 or more; its first real use removed 9.3% of a published corpus (subsection 3.1). The meaning filter admits LLM-written pairs only if they keep the source's meaning. Generated pairs pass validators before they may be trained on. The locked test sets are fixed by row count and SHA-256 in a committed manifest, re-verified before every test run.

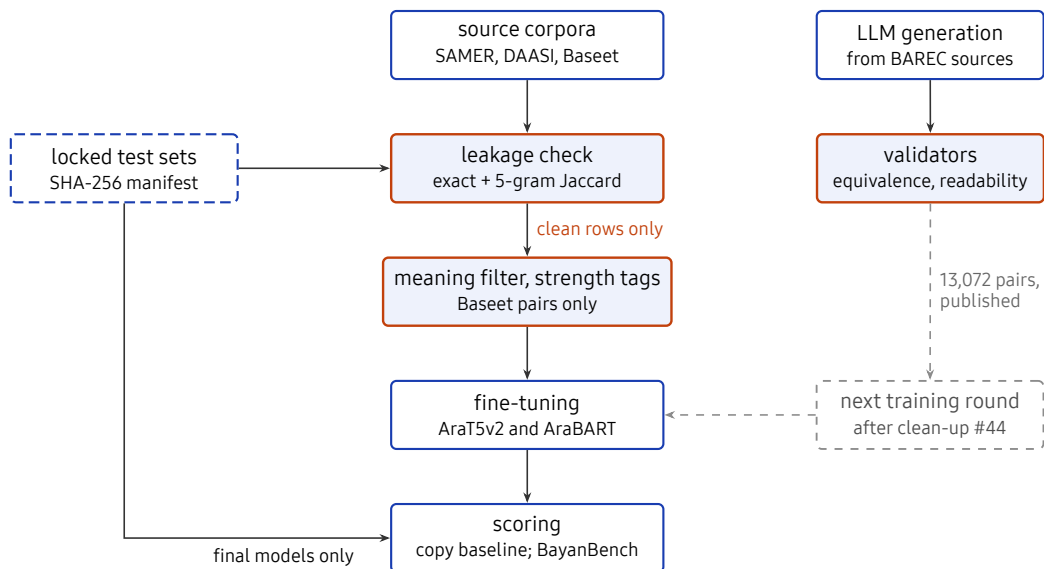


Figure 2. Data and evaluation workflow. Orange boxes are the gates that can reject data. The generated corpus (right) is published and waits for the clean-up in #44 before it enters training.

2.4. System Design

Figure 3 shows the system as it runs on a phone today. The Android application registers an activity for `android.intent.action.PROCESS_TEXT` (text/plain, Android 6.0 and later), so تبسيط appears in the selection menu of any app. A translucent activity draws the result over the host app without the overlay permission; an optional floating panel, which does need it, keeps reading while the reader returns to the page.

Inference. The simplifier ships as an int8 ONNX bundle — encoder, decoder with a key-value cache, and the tokenizer — downloaded once and checked by SHA-256, then run by ONNX Runtime with a greedy decoding loop written in Kotlin (no repeated 3-grams, at most 256 tokens). The app strips tashkeel, splits the selection into sentences, and decodes sentence by sentence, streaming each token to the screen as it is produced. Sentences under six words (headings, labels) are passed through unchanged, because the model was trained on longer sentences and invents text around fragments. Every output then passes a *retention fallback*: if the rewrite keeps less than 35% of the source’s words, the source sentence is shown instead. The original is always one tap away.

Read-aloud. Voices run on the device through sherpa-onnx. The default is Nabra-7M-Distill, a Kokoro-architecture Arabic voice exported to int8 (8.7 MB); three Piper voices are optional downloads. The spoken word is highlighted in the text as it is read.

Diacritization. Libtashkeel was selected on the benchmark in Table 16 (4.8 MB, MIT). It is not yet wired into the app, and whether full tashkeel helps our readers is itself an open question (subsection 1.1); it will be offered as a reader setting, off by default.

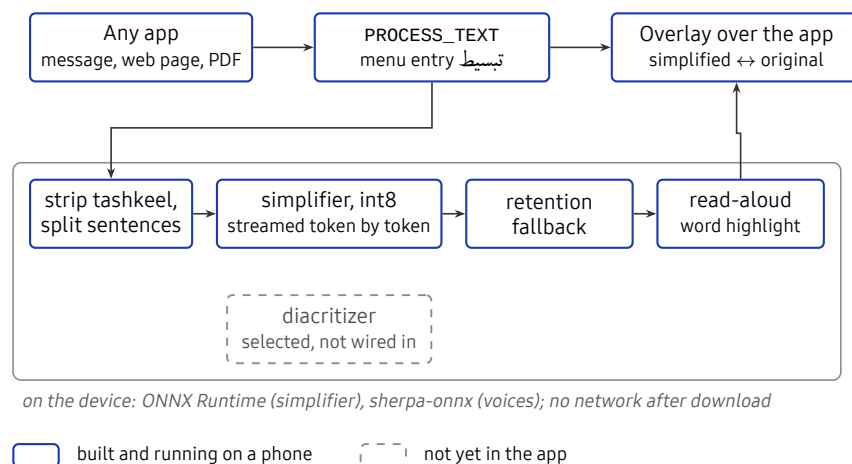


Figure 3. System architecture on 28 September. The plugin is entered from the host app’s selection menu and answers with an overlay on top of it.

2.5. BayanBench: a Benchmark for the Reader’s Task

SARI measures agreement with one editor’s rewrite. Our readers need something else: that a selection comes back *easier to decode* and *with the same meaning*, and that what should not change does not. No public benchmark tests this for Arabic, so we built one, following the construct-validity checklist of [22] and the reporting practices of [23].

Construct. A rewrite is dyslexia-friendly when it says the same thing in a form that is easier to decode and follow: no clause over 15 words (BAREC’s guideline cap for level 10, about grade 4), more familiar words, no new errors, and restraint where the text is already easy. Out of scope:

visual formatting, read-aloud, tashkeel, and summarising — dropping content always counts as a failure, never as a simplification.

Tracks. Each track targets one behaviour and is scored on its own (Table 6). Probe tracks follow CheckList [21]: fixed behaviours checked item by item.

Table 6. BayanBench tracks. Items per track in the dev / test splits.

Track	Behaviour	Measured by	Items
A1–A3	meaning kept: general; hard spots (negation, numbers, conditions, cause, quantity, certainty); high stakes (legal, medical)	judge (insertion, deletion, substitution, severity [18]); code (numbers, negations)	299 / 508
B	long clauses fixed	code: no clause over 15 words	104 / 187
B2, B3	fewer hard words; lower reading level	SAMER word levels 4–5; CAMEL BAREC readability model	across items
C	easy text left unharmed	code: unchanged; judge	129 / 202
D	correct Arabic	judge; house punctuation rules	across items
E1–E4	real selections: cut-offs, messy text, 40+ words, several sentences	code (end mark, clause length); judge	139 / 263
F	protected text (Qur’an, Bible, classical poetry)	exact letters	60 / 90
G	on the phone	int8 bundle vs the same model in PyTorch; size, time	—
H	people	blind ratings, 100 items (prepared)	—

Items and splits. 1,981 items in four sets that are always reported side by side, never averaged: *core* (BAREC test documents), *held-out* (news and legal text, plus 247 BAREC sentences of those domains that no model has trained on), *outside* (344 sentences from UN News, WHO fact sheets, newly created Arabic Wikipedia articles, UNHCR help pages and Android help, fetched on 28 September and so published after every corpus we use), and *written* (50 medicine-leaflet sentences written for the benchmark by an LLM, kept out of core because they are not real text). The split is by document — 731 dev items from 129 documents, 1,250 test items from 197 — so no document contributes to both. Any BAREC document with a sentence found in any training or generated data was dropped whole (17 documents), and near-copies were removed by word-trigram overlap.

Two tiers. The *code tier* is deterministic: numbers, negations, clause length, end marks, house rules, hard words, reading level, unchanged text and protected letters. Anyone gets the same numbers anywhere. The *judge tier* holds the meaning and grammar verdicts, given by Gemini 3.1 Pro with a fixed Arabic checklist; the judge sees only the selection and the output, never the system’s name. Verdicts are stored as shared data keyed by the content of the (selection, output) pair, so scoring never calls a judge, an unjudged output is reported as *pending* rather than guessed, and a judge-tier rate is shown only once 90% of its items are judged.

Trusting the judge. Before use we planted seven kinds of error (a number changed, a negation dropped, a quantifier swapped, a clause deleted, a detail invented, a mid-sentence full stop, broken agreement) in outputs the judge had passed. It flagged a major error in all 47, named the planted type in 45, and gave the same pass/fail verdict on 12 of 13 pairs judged twice. A new judge is accepted only if its verdicts on a 792-pair validation set clear thresholds that were

fixed *before* any candidate was tested (recall of meaning errors ≥ 0.80 , pass rate on clean pairs ≥ 0.70 , $\kappa \geq 0.60$, planted errors $\geq 90\%$, recall of language errors ≥ 0.60). An open-weight candidate, Qwen3.8-27B run on Kaggle, reached 0.82 recall on meaning errors but $\kappa = 0.49$ and only 0.26 recall on language errors, so it was not accepted. Model 2's training data contains nothing the judge wrote or approved, and the benchmark's written items and key questions come from a different model family, to limit preference leakage [25].

Statistics and release. Every rate carries a 95% interval from a bootstrap over whole documents, because sentences from one document pass or fail together [24]; a comparison counts only when the interval of the paired difference excludes zero. The benchmark ships as a Python package (`bayanbench verify | predict | score | pending | judge | validate-judge`) with its data in a private Hugging Face dataset pinned by checksums, so any team member can score a model on a laptop, on Kaggle or on Linux without us.

3. Results

3.1. Data Preprocessing

Conversion. SAMER's aligned levels were converted to source–target pairs (level 5 $\rightarrow$ level 3). One converter (`make_eval_input.py`) turns each locked test file into the scoring format, keeps every row — including the 1,599 SAMER test rows (48.8%) where the two levels are identical — and checks the row count before writing.

Normalisation for comparison only. For leakage checks and scoring alignment, diacritics and punctuation are removed and alef, ؤ and ي/ي variants are unified. Training text keeps its original spelling apart from tashkeel and tatweel.

Leakage checks. Every training file — model 1's, model 2's and the synthetic export — was checked against both locked test sets and found clean. Matches under five words are reported but not counted, because short formulaic fragments recur in any corpus; BAREC alone repeats 311 sentences between its own train and test splits.

Contamination found and removed. 3,190 of Baseet's 34,294 rows (9.3%) overlapped our locked test sets — 1,829 SAMER test sentences and 1,361 BAREC test sentences. Seen the other way, 47% of SAMER's substantive test sentences appear somewhere in Baseet. Training on it unfiltered would have inflated every later score without any visible error. The rows were removed and the remaining 31,104 re-checked: zero matches.

The meaning filter. Applied to the Baseet candidates, the filter keeps 6,888 pairs across the three levels. Its purpose shows at the strongest level: the 5th percentiles of Baseet's level-1 targets are 0.44 LaBSE similarity and 26% of the source's length, and 10% of them drop a number, against 0.646, 67% and none for DAASI's human rewrites.

3.2. Exploratory Data Analysis (EDA)

We measured the corpora rather than assume them (Table 7, Figure 4); the differences explain both models' behaviour.

- **SAMER has no diacritics at all**, while 54% of BAREC test and 36% of Baseet carry them. A model trained only on SAMER has never seen a diacritized input.

Table 7. Corpus statistics, computed on the files we hold. Identity = target equals source after removing diacritics; length ratio = target words over source words, median.

Corpus	Rows	Median words	≤ 3 words	Diacritized	Identity	Length ratio
SAMER train	14,343	7	16.4%	0.0%	42.1%	1.00
SAMER test (locked)	3,277	6	23.6%	0.0%	48.8%	1.00
BAREC test (locked)	7,286	10	16.9%	53.8%	—	—
Baseet, strongest level	31,104	14	0.0%	36.5%	0.1%	0.73
DAASI	1,509	12	8.4%	32.3%	0.0%	1.00

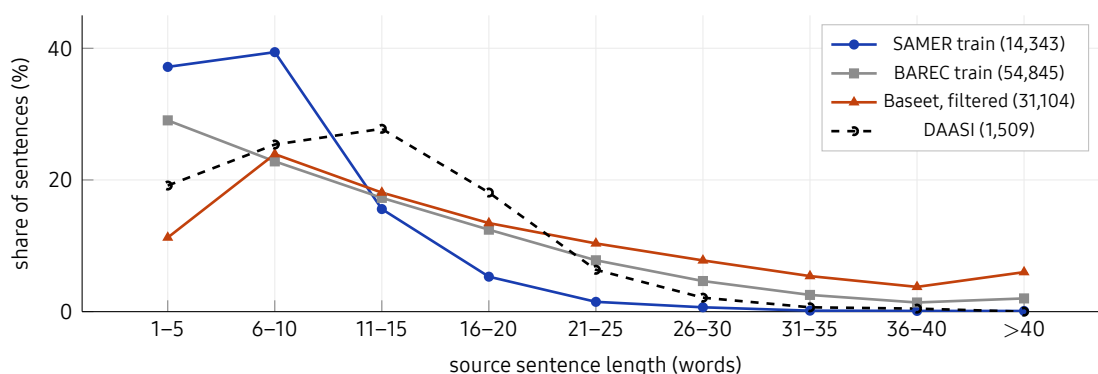


Figure 4. Source sentence length by corpus. SAMER is short literary text; Baseet and BAREC reach the long sentences a reader actually struggles with.

- **SAMER simplifies words, not sentences.** Its median length ratio is exactly 1.00: humans substituted vocabulary and kept structure. Baseet’s strongest level shortens a 14-word median source to 9 words (ratio 0.73).
- **No corpus teaches splitting.** A sentence is added in 0% of SAMER pairs, 2–4% of Baseet’s and 8% of DAASI’s, and word length and clitic load barely move in any of them. Existing data can teach “shorter, more common words”; splitting long sentences — the change the reading research supports most directly [6] — has to come from new data.
- **Half of SAMER needs no change.** 42–49% of pairs are identical, and SAMER splits by chapter rather than by novel, so its test set is fully in-domain. Both make SARI on SAMER read optimistically.

3.3. Modeling

The baseline comes first. Because half of SAMER needs no change, copying the input scores well: SARI 77.50 on the full test set, but only 56.06 on the 1,678 rows a human changed. The second number is the bar a model must clear. BAREC has no reference simplifications, so on it only reference-free measures apply.

Model 1 on the locked test sets. We ran model 1 once on each locked test set, with the task prefix it was trained on, under greedy decoding (what the phone runs) and beam search with four beams (Table 8, Figure 5). Before the test run, the harness reproduced the training run’s 2,983 dev predictions exactly, row for row.

Table 8. Model 1 on test data with human references, scored once with `score.py`. SARI is split by whether the human reference changed the source (DAASI has no unchanged pairs). BERTScore compares output with source; readability drop is the fall in predicted BAREC level (positive = easier). DAASI sources and references have tashkeel stripped.

System	SARI all	SARI changed	SARI un-changed	BLEU	BERT-Score	Readability drop	Copy rate
SAMER test (3,277 rows, locked, in domain)							
Copy the input	77.50	56.06	100.00	72.03	1.00	0.00	100.0%
Model 1, greedy	75.88	61.83	90.62	71.68	0.98	0.08	65.3%
Model 1, beam 4	76.29	61.95	91.34	72.22	0.98	0.07	69.4%
DAASI held-out (350 pairs, government and insurance text, out of domain)							
Copy the input	45.05	45.05	—	25.25	1.00	0.00	100.0%
Model 1, greedy	43.87	43.87	—	23.66	0.99	-0.03	77.7%

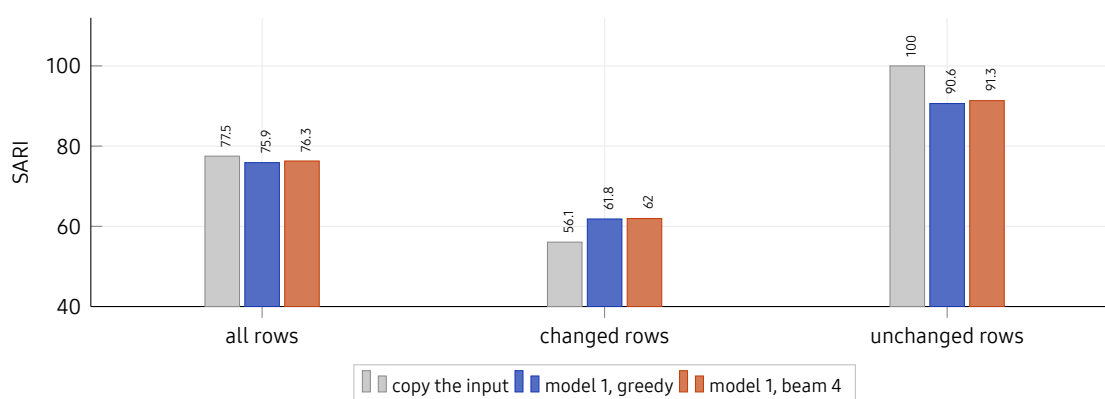


Figure 5. SARI on the locked SAMER test set by reference bucket. Model 1 clears the copy baseline where a human simplified, and loses to it where a human left the sentence alone.

Model 1 simplifies where it should, and over-edits where it should not. On the 1,678 test rows a human simplified, greedy decoding scores 61.83 against the copy baseline’s 56.06 (+5.77 SARI). On the 1,599 rows a human left alone it scores 90.62 against a perfect 100, because it edits 13.6% of sentences that needed no change. The two effects net to -1.62 overall. Beam search does not change the picture (61.95 on changed rows), so greedy decoding costs nothing on the phone.

It learned what SAMER teaches. The model returns 65.3% of test rows unchanged where humans left 48.8%, and leaves 45.3% of the rows a human rewrote untouched. When it edits, 75.4% of its edits change a single word (humans: 49.5%), and its median output-to-input length ratio is 1.00: it substitutes vocabulary and never restructures (Figure 7a). Figure 6 shows how checkpoint selection contributed: the SARI that chose the checkpoint had been flat within noise since step 1,800 while the copy rate was still falling.

Stripping tashkeel is what keeps it usable out of domain. On BAREC’s 3,918 test sentences that carry tashkeel, raw input keeps only 63.7% of the source words and 31.9% of outputs lose more than half the sentence; stripped, the same sentences keep 94.6% and only 1.8% lose more than half (Table 9, Figure 7b).

Out of domain, it does not beat copying. On DAASI’s held-out government and insurance text, model 1 scores 43.87 against a copy baseline of 45.05 (-1.19) and returns 77.7% of sentences unchanged: it learned from SAMER’s novels when *not* to edit, not how to simplify administrative

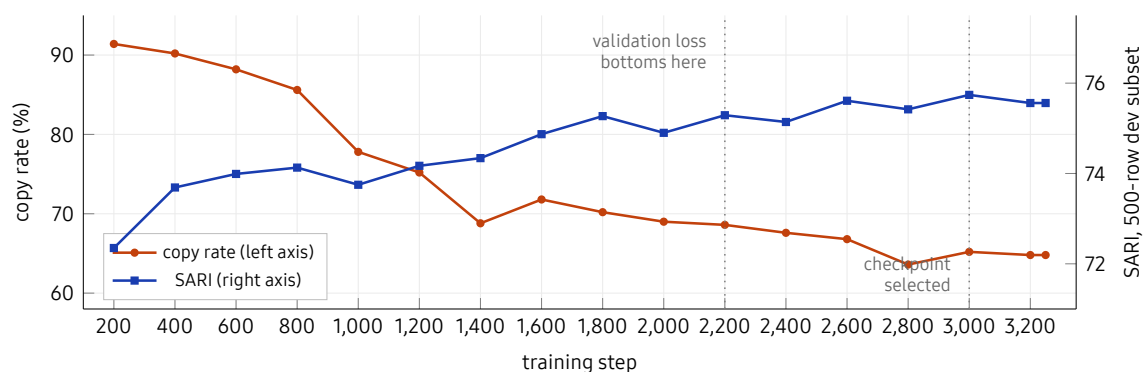


Figure 6. Model 1 training. The checkpoint was selected on a 0.18-point SARI difference while the model’s copying behaviour was still changing.

prose.

Table 9. Model 1 on the locked BAREC test set (7,286 sentences, no reference simplifications), greedy decoding. Stripped: tashkeel removed from the source before the model sees it, as the application does.

Input	Copy rate	BERTScore	Readability drop	Source words kept	Rows losing >half
Stripped (as deployed)	68.6%	0.98	−0.04	94.9%	1.4%
Raw, with tashkeel	37.5%	0.91	−0.45	78.2%	17.5%

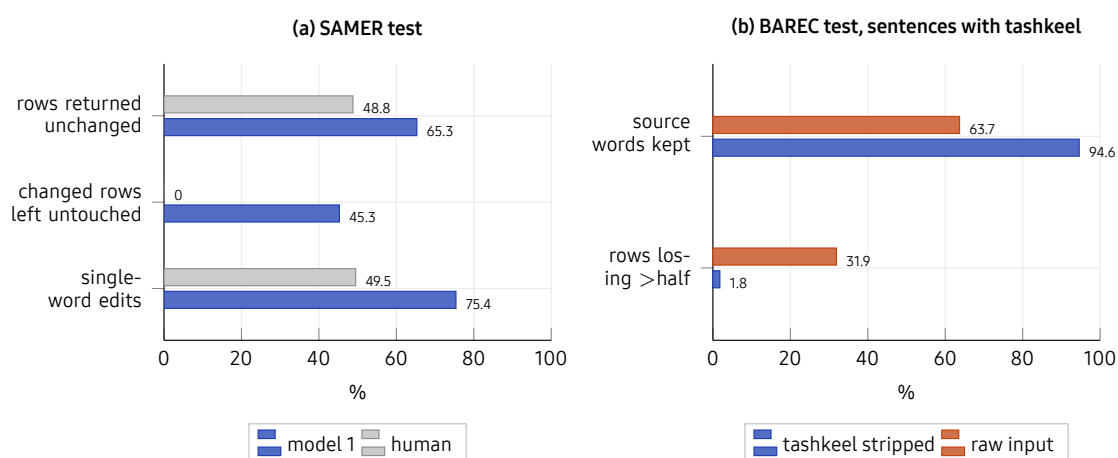


Figure 7. Model 1 behaviour. (a) SAMER test, greedy, against the human reference. (b) BAREC test, the 3,918 sentences that carry tashkeel, greedy, with and without tashkeel stripped from the input.

Meaning errors are not rare. On dev, 832 of model 1’s substitutions disagree with the human, 347 of them on words the human deliberately kept; confirmed examples include عزلة (isolation) → استعباد (enslavement) and ملجم (bridled) → مسلح (armed). SARI and BLEU do not detect this class of error. This finding is the reason BayanBench exists.

Model 2 training. Table 10 and Figure 8 summarise the two runs. Both clear model 1’s selection score comfortably: for reference, model 1 scored 63.66 SARI on SAMER dev’s changed rows (all 1,754 of them, a superset of the 600 used here). AraT5v2 was still improving at its last evaluation; AraBART peaked at epoch 9 and was flat from epoch 6. Copying falls from 80–91% to about 51% of rows for both, against 33.3% for the human references on the same rows, so over-conservatism is reduced, not solved: AraT5v2 still returns 186 of the 600 changed rows unchanged.

Table 10. Model 2 runs on one T4 GPU. SARI and copy rate on the 900-row selection sample (600 changed, 300 unchanged SAMER dev rows, tag [s0]). Speed: sentences per second on SAMER test sources, fp16, batch 64, greedy, same GPU and run (#35).

	AraT5v2	AraBART
Parameters	368M	139M
Training time	6,783 s	2,600 s
Best checkpoint (step of 14,210)	14,200	12,780
SARI, 600 changed rows	66.58	64.71
SARI, all 900 rows	75.64	73.62
Copy rate, 900 rows (human: 33.3%)	51.0%	51.4%
Generation speed (sentences/s)	80.8 [†]	292.8
[†] measured on model 1, the same architecture and size.		

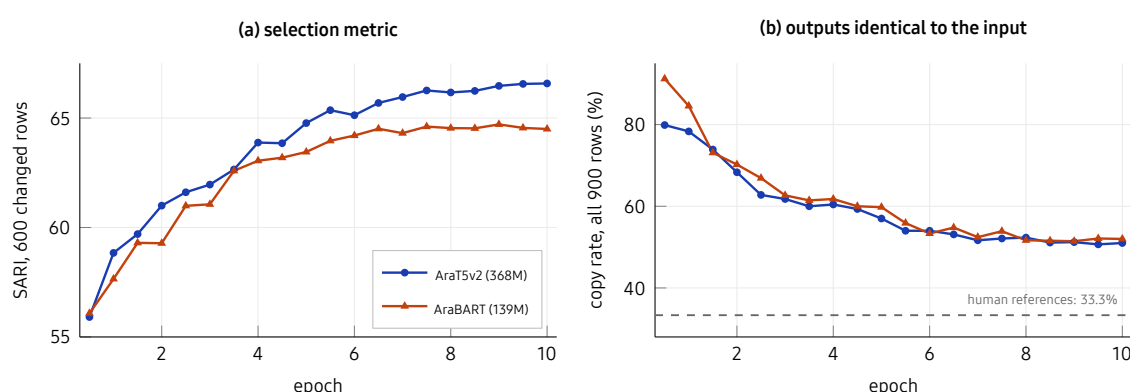


Figure 8. Model 2 training, from the committed notebooks: (a) the selection metric at each of the 20 evaluations; (b) the share of outputs identical to the input, with the human references’ rate as a dashed line.

Model 2 on BayanBench. We ran both models once on the BayanBench test split, in the medium setting ([s2]), and also ran the two int8 bundles exactly as the app downloads and calls them (Table 11; dev in Table 14). The judge tier is being filled: after the 28 September re-split, fewer than 90% of model 2’s outputs have verdicts, so, by the benchmark’s own rule, no judge-tier rate is reported here. The code-tier results support five findings.

F1 Model 2 makes text easier, by the measures the reading research supports. In the [s2] setting, every clause of a long-clause item ends up at 15 words or fewer in 50.8% (AraT5v2) and 55.1% (AraBART) of B items, against 0% for copying; hard words fall in 41.2–44.0% of items that have any, and the predicted reading level falls in 20.0–21.2%. The two architectures are statistically indistinguishable on every easier measure.

F2 It keeps most of the content, and AraT5v2 keeps more. AraT5v2 keeps at least 60% of the source’s words in 88.2% of items, AraBART in 81.7%, a paired difference whose interval excludes zero. House punctuation rules are broken in about 1% of outputs.

F3 It does not know when to stop. Easy text, which should come back unchanged, does so in only 9.4% (AraT5v2) and 11.4% (AraBART) of C items; protected text (Qur’anic verses, classical poetry) keeps its letters in only 4.4%; and a selection cut mid-sentence keeps its ending in about a third of E1 items — the rest are “completed” with a full stop. The training data never shows these behaviours: SAMER’s unchanged pairs are literary sentences, not easy prose, and no corpus contains protected or cut-off text.

F4 Numbers and negations need a guard. Numbers survive in 76.3% of items that contain

Table 11. Model 2 on the BayanBench **test** split (1,250 items, 197 documents), run once, code tier. Pass rates in %, with the 95% interval from a bootstrap over whole documents in brackets. The first two model columns send the [S2] (medium) strength tag the model was trained with; the two *int8 app* columns are the quantised bundles the app downloads, which send no tag. All outputs pass through the app’s retention fallback and punctuation fix. Bold: the best model on each row. *n* = items the measure applies to.

Measure (pass rate, %)	Copy the input	AraT5v2, [S2]	AraBART, [S2]	AraT5v2, int8 app	AraBART, int8 app	<i>n</i>
Easier						
No clause over 15 words (B, long clauses)	0.0	50.8 [44–57]	55.1 [48–62]	18.7 [13–24]	43.9 [38–50]	187
No clause over 15 words (E3, 40+ words)	25.0 [14–36]	46.9 [35–57]	50.0 [39–60]	18.8 [9–29]	34.4 [22–48]	64
Fewer hard words (items with any)	0.0	41.2 [37–45]	44.0 [39–49]	24.3 [21–28]	43.0 [39–47]	588
Lower reading level (sources at level 10+)	0.0	20.0 [17–23]	21.2 [18–24]	8.2 [7–10]	15.3 [13–18]	898
Faithful						
Kept 60%+ of the source’s words	100.0	88.2 [86–90]	81.7 [78–85]	98.4 [98–99]	91.5 [89–94]	958
Numbers kept (items with numbers)	100.0	76.3 [56–94]	74.0 [57–89]	75.7 [55–93]	71.0 [51–88]	169
Negations kept (items with a negation)	100.0	79.3 [74–84]	80.6 [75–86]	92.4 [89–95]	84.4 [80–88]	237
No house-rule break	100.0	99.1 [98–100]	99.0 [98–100]	99.2 [99–100]	99.1 [99–100]	1,160
Restraint						
Easy text left unchanged (C)	100.0	9.4 [6–13]	11.4 [7–16]	33.2 [26–40]	19.8 [14–26]	202
Cut-off selection keeps its ending (E1)	100.0	35.1 [25–46]	37.2 [26–49]	37.2 [28–48]	43.6 [33–55]	94
Protected text: letters unchanged (F)	100.0	4.4 [0–10]	4.4 [0–11]	47.8 [30–70]	24.4 [16–36]	90

one, negations in 79.3%. Most number failures (24 of 40) are clause markers such as “(1)” in the human-rights declaration, which a rewrite drops harmlessly. The rest include a real, fixable fault: digits in the Persian range (۱۹۹۲) are dropped or replaced, because the tokenizer has barely seen them.

F5 The app currently sends no strength tag. The shipped bundles send the bare prefix, and Table 12 shows the cost: without the tag the model is safer (more words kept, more easy text left alone) but fixes far fewer long clauses. Quantising to int8 changes little for AraBART; for AraT5v2, whose untagged int8 output is the most conservative system we measured, it shifts behaviour further toward copying. The strength tag belongs in the app’s request.

Across item sets. Table 13 reports the two systems a reader would meet — AraT5v2 in the medium setting and the int8 AraBART the app ships — on each item set. The easier measures are lower away from core: AraT5v2 fixes long clauses in 58.9% of core items but 39.1% of held-out and 40.4% of outside items, so part of the core result reflects the genres of BAREC’s documents. The gains nonetheless stay far above copying on text published after every corpus we use. Restraint is weak everywhere, and weakest on held-out and outside text, where fewer than 7% of easy sentences come back unchanged.

Table 12. Paired effects on the test split, in points, with 95% document-bootstrap intervals; * = the interval excludes zero. Same items, same model, one change at a time.

Change	No clause over 15 (B)	Fewer hard words	Kept 60%+ of words	Easy text unchanged (C)
AraT5v2: drop the [s2] tag	-13.4* [-19, -8]	-7.1* [-10, -4]	+7.0* [+5, +9]	+9.9* [+6, +14]
AraBART: drop the [s2] tag	-11.2* [-18, -5]	-2.7 [-6, +1]	+11.5* [+9, +14]	+8.4* [+4, +14]
AraT5v2: PyTorch fp32 → int8 ONNX	-18.7* [-25, -12]	-9.7* [-12, -7]	+3.2* [+2, +5]	+13.9* [+9, +19]
AraBART: PyTorch fp32 → int8 ONNX	+0.0 [-3, +3]	+1.7* [+0, +3]	-1.7* [-3, -0]	+0.0 [-3, +3]

Table 13. Model 2 on the four BayanBench item sets (test split), pass rate in %, number of items in brackets. *Core*: BAREC test documents; *held-out*: news and legal text; *outside*: 344 web sentences published after every corpus we use; *written*: medicine-leaflet sentences. The sets are reported side by side and never averaged.

System and measure	Core	Held-out	Outside	Written
AraT5v2, [s2]				
No clause over 15 words (B)	58.9 (107)	39.1 (23)	40.4 (57)	—
Fewer hard words	44.8 (328)	34.1 (129)	37.4 (123)	62.5 (8)
Lower reading level	22.4 (545)	20.9 (163)	13.2 (167)	8.7 (23)
Kept 60%+ of words	88.3 (582)	86.8 (182)	88.3 (171)	95.7 (23)
Easy text left unchanged	14.3 (77)	6.5 (92)	6.1 (33)	—
AraBART, int8 app				
No clause over 15 words (B)	43.0 (107)	30.4 (23)	50.9 (57)	—
Fewer hard words	45.7 (328)	37.2 (129)	43.1 (123)	25.0 (8)
Lower reading level	16.0 (545)	16.6 (163)	12.6 (167)	8.7 (23)
Kept 60%+ of words	93.5 (582)	89.0 (182)	87.1 (171)	95.7 (23)
Easy text left unchanged	24.7 (77)	18.5 (92)	12.1 (33)	—

3.4. On the Device

Table 15 reports what the phone pays. Both simplifiers were exported to ONNX with the tokenizer inside the bundle and quantised to int8. The AraBART bundle meets the 250 MB budget; the AraT5v2 bundle does not, because 46% of its weights are two untied 110k-row embedding matrices that we have not yet pruned.

Because the output streams, what the reader waits for is the first token, not the whole sentence: under a third of a second on a phone that is now a mid-range device. On the phone we have measured only the AraT5v2 bundle so far; AraBART is 1.9 times faster on the laptop with the same loop, and its phone measurement is scheduled with the release build (#31).

Diacritization. Four systems were benchmarked on the 743-sentence Tashkeela test fixture (Table 16). Libtashkeel was selected: CATT is statistically tied on accuracy (0.01 points, about six characters in 67,576) but 16 times larger and carries a non-commercial licence. CAMEL Tools was rejected for a stronger reason than accuracy: it changed the base letters of 32 of 100 sentences, corrupting the text it was asked to vocalise.

3.5. User Interface

Figure 9 shows the interface on a phone. Selecting Arabic text in any app offers تبسيط; choosing it opens the result over the app, the simplified text appearing sentence by sentence as it is de-

Table 14. The same measures on the BayanBench **dev** split (731 items, 129 documents), for comparison with Table 11.

Measure (pass rate, %)	Copy the input	AraT5v2, [s2]	AraBART, [s2]	AraT5v2, int8 app	AraBART, int8 app	<i>n</i>
Easier						
No clause over 15 words (B, long clauses)	0.0	56.7 [46–68]	62.5 [52–72]	17.3 [11–24]	45.2 [35–55]	104
No clause over 15 words (E3, 40+ words)	38.7 [21–56]	41.9 [25–60]	48.4 [31–67]	32.3 [16–52]	48.4 [29–69]	31
Fewer hard words (items with any)	0.0	42.9 [36–49]	42.4 [36–49]	27.1 [21–32]	40.6 [35–47]	347
Lower reading level (sources at level 10+)	0.0	18.4 [15–22]	17.4 [14–21]	4.9 [3–7]	14.9 [11–19]	511
Faithful						
Kept 60%+ of the source’s words	100.0	92.8 [91–95]	83.9 [81–87]	99.1 [98–100]	91.1 [89–93]	542
Numbers kept (items with numbers)	100.0	86.9 [77–95]	83.6 [73–92]	88.5 [78–96]	88.5 [78–96]	61
Negations kept (items with a negation)	100.0	86.2 [79–93]	85.3 [78–92]	95.4 [92–99]	90.8 [84–96]	109
No house-rule break	100.0	99.0 [98–100]	99.0 [98–100]	99.3 [99–100]	99.6 [99–100]	671
Restraint						
Easy text left unchanged (C)	100.0	14.0 [8–20]	14.0 [9–19]	37.2 [28–46]	23.3 [17–29]	129
Cut-off selection keeps its ending (E1)	100.0	35.7 [23–47]	42.9 [31–55]	44.6 [33–55]	50.0 [37–62]	56
Protected text: letters unchanged (F)	100.0	11.7 [4–26]	6.7 [0–16]	48.3 [21–91]	26.7 [11–55]	60

coded. A single toggle switches between *Simplified* and *Original*; below it are read-aloud, copy and share. While Bayan reads aloud, the current word is highlighted. The panel can float over the page or shrink to a small bar, so the reader keeps the host app in view while listening. The app follows Android’s own Material 3 design rather than a design system of its own, and lays text out right to left without justification, because justification in Arabic stretches words and opens uneven gaps. Settings hold the reader’s typography — typeface, size (22 sp by default), line height (1.8) and word spacing, the one spacing adjustment that carries over to cursive Arabic, where letter spacing breaks the joins — the background, the simplification model, the voice and its speed, and whether words are highlighted while reading.

3.6. Testing and Improvements

Testing. The original plugin ships 41 unit and Robolectric tests, and the download pipeline in PR #37 checks every file by SHA-256. The leakage checker is tested against a planted duplicate and must exit non-zero. The benchmark package has its own tests on a built-in toy dataset, and its prediction command reproduces our model-2 outputs on all 731 dev items. Three harness faults were found by running things end to end on real data rather than fixtures: the scoring script put every BAREC row in the “unchanged” bucket; the shared prediction script omitted model 1’s training prefix, which alone cost twelve SARI points; and length-grouped batching

Table 15. On-device cost. Laptop: 4 CPU threads, median and 90th percentile per test sentence, the app's own decoding loop (BayanBench track G). Phone: Xiaomi Mi 11X (Snapdragon 870, Android 13, 6 GB RAM), the app's logs over six runs on a two-sentence paragraph.

	AraBART, int8	AraT5v2, int8
Download (encoder + decoder + tokenizer)	222 MB	471 MB
Laptop CPU, per sentence (median / p90)	0.10 / 0.24 s	0.20 / 0.43 s
Phone: model load, cold	—	2.6–7.3 s
Phone: first token, first sentence	—	177–318 ms
Phone: first token, later sentences	—	55–85 ms
Phone: whole sentence	—	0.48–2.24 s
Read-aloud voice (Nabra-7M-Distill, int8)	8.7 MB	

Table 16. Diacritization benchmark (#23). DER = diacritic error rate; base-letter corruption measured on 100 sentences.

System	DER	DER, no case	WER	Base letters changed	Size	Licence
CAMeL Tools MLE [33]	16.57%	13.97%	56.73%	32%	129 MB	MIT/GPL
CATT [34]	8.44%	6.92%	29.40%	0%	77.9 MB	CC BY-NC
Mishkal [35]	12.74%	10.12%	39.33%	0%	4.3 MB	GPLv3
Libtashkeel [36]	8.43%	6.91%	29.57%	0%	4.8 MB	MIT

shuffled the evaluation set. Each now has a guard.

Improvements. Each measured problem produced a change (Table 17).

4. Projected Impact

4.1. Accomplishments and Benefits

Built so far. A working Android app on the `PROCESS_TEXT` intent, with on-device simplification, streaming and read-aloud; two model-2 simplifiers in the team's Hugging Face organisation, trained on one shared script and one leakage-checked, meaning-filtered dataset; a published 13,072-pair synthetic corpus; a selected diacritizer with a four-system benchmark behind it; and a benchmark, with its package and judge protocol, that the team can run without us.

Benefits. For readers: help at the moment and place the difficulty occurs, offline and private, with the original always one tap away. For schools and teachers: a tool that works on the phones students already own. For the field: a documented case of test-set contamination in a public Arabic simplification corpus, an evaluation protocol that exposes copy-heavy scores, and a benchmark that measures what an Arabic reader with dyslexia needs rather than agreement with one editor.

4.2. Future Improvements

Before submission (5 October). Send the strength tag from the app and add the three input guards of Table 17 (protected text, cut-off endings, digit ranges); finish the judge tier and the SARI tables (#30); measure AraBART on the phone with the release build (#31); run the human evaluation with readers.

Next training round. Train on the synthetic corpus after the clean-up in #44, which is the first



(a) result sheet over the web page, reading aloud

(b) floating panel, the page stays usable

(c) minimised: reading continues

Figure 9. The app on a phone (Snapdragon 870, Android 13), over an Arabic Wikipedia page. (a) The result sheet, reading aloud with the current word highlighted. (b) The floating panel. (c) Minimised to a bar while reading continues.

data we hold where most pairs split long sentences (84%). Select the checkpoint on Bayan-Bench dev instead of SAMER SARI, so that the behaviours the reader needs enter the choice.

Beyond the capstone. Vocabulary pruning to bring AraT5v2 under budget; a reading study measuring speed and comprehension with and without simplification; tashkeel as an evidence-tested reader option.

4.3. Limitations

- **No reader has yet used Bayan.** Every result here is automatic. The measures are chosen from reading research, but none is validated as a predictor of reading success for Arabic readers with dyslexia; no such metric exists in any language.
- **The 15-word clause limit is a design choice.** It matches BAREC’s guideline for level 10, and the ranking of systems is the same at 12, 15 and 20 words, but the absolute rates move with it.
- **The judge is one model.** It passed our planted-error and self-consistency checks, and it is kept away from data it produced, but LLM judges are least consistent in lower-resource languages, Arabic among them.
- **Some tracks are small.** E3 and E4 hold 64 and 40 test items and can separate only large differences.
- **One phone.** On-device figures come from one mid-range phone and, so far, one of the two bundles.

4.4. Ethical Considerations

A simplification that changes the meaning can mislead exactly the reader who is least able to check it. Bayan therefore always shows the original one tap away, falls back to the source

Table 17. What the measurements changed.

Finding	Change	Status
A copying model scores SARI 77.5 on SAMER test	Copy baseline and changed/unchanged split reported with every score	Done
9.3% of Baseet overlaps our test sets	Rows removed; leakage check required for every training file	Done
Model 1 collapses on diacritized input	Tashkeel stripped before the simplifier, in training and in the app	Done
Model 1 chosen on SARI noise over a copy-heavy dev set	Model 2 selects on changed-row SARI only	Done
Model 1 edits 13.6% of sentences that need no change, yet copies 65%	All unchanged pairs kept in model-2 training	Done
SAMER teaches word swaps only; model 1 below copying on DAASI	DAASI and meaning-filtered Baseet added, each under its own tag	Done
Prediction script omitted the training prefix	Prefix a required setting of every model and bundle	Done
Length grouping shuffled the evaluation set	Grouping off; alignment check stops the run	Done
SARI is blind to meaning errors	BayanBench, with a validated judge and code checks	Done
Model 2 edits easy text and protected text (F3)	Protected-text detection in the app; easy prose and scripture as unchanged training pairs	Not started
Cut-off selections are completed (F3)	Keep the source's end mark in the app; cut-off pairs in the next corpus (#44)	Not started
Persian-range digits dropped (F4)	Map them to Arabic-Indic digits before the model, restore after	Not started
The app sends no strength tag (F5)	Send the reader's strength tag with every request	Not started
AraT5v2 int8 bundle is 471 MB	Vocabulary pruning, or ship AraBART	In progress
No training data teaches splitting	Next round on the cleaned synthetic corpus (84% of its pairs split)	In progress

sentence when a rewrite drops most of its words, and is evaluated on meaning before ease. Religious text is protected on the benchmark and will be protected in the app. The reader's text never leaves the phone. SAMER is used under CAMEL Lab's written approval and never redistributed; the human evaluation will run with informed consent. The tool is an aid, not a diagnosis or a treatment.

References

- [1] A. F. Aldakhil. Prevalence of developmental dyslexia among primary school children in Arab countries: a systematic review and meta-analysis. *Research in Developmental Disabilities*, 152:104812, 2024.
- [2] E. Saiegh-Haddad and H. Taha. The role of morphological and phonological awareness in the early development of word spelling and reading in typically developing and disabled Arabic readers. *Dyslexia*, 23(4):345–371, 2017.
- [3] L. Rello, R. Baeza-Yates, L. Dempere-Marco and H. Saggion. Frequent words improve readability and short words improve understandability for people with dyslexia. *INTERACT*, pp. 203–219, 2013.
- [4] M. Rivero-Contreras, P. E. Engelhardt and D. Saldaña. An experimental eye-tracking study of text adaptation for readers with dyslexia: effects of visual support and word frequency. *Annals of Dyslexia*, 71:170–187, 2021.

Table 18. Objectives set in subsection 1.2, against the evidence on 28 September.

Objective	Evidence so far	Status
O1 Beat copying on changed rows; easier text without losing meaning	Model 1: 61.83 vs 56.06 on SAMER test (+5.77). Model 2: 66.58 on changed dev rows; on BayanBench test, long clauses fixed in 50.8% and 55.1% of items, $\geq 60\%$ of words kept in 81.7–88.2%. Test-set SARI tables and the judge tier pending (#30)	In progress
O2 Diacritize without changing letters	Libtashkeel: DER 6.91% without case endings, 0 of 100 sentences altered, 4.8 MB; not yet in the app	In progress
O3 Run on a phone within 250 MB	AraBART int8: 222 MB, in the app; AraT5v2 int8 on a Snapdragon 870: first token in 55–318 ms	Done
O4 Evidence from readers	Blind 100-item rating set and rating page prepared; sessions not yet run	Not started

- [5] I. A. Asadi, A. Asli-Badarneh and T. Vaknin-Nusbaum. The impact of morphological density on reading comprehension in Arabic: a comparison between typically developing and children with reading disabilities. *Dyslexia*, 30(1):e1761, 2024.
- [6] L. Javourey-Drevet, S. Dufau, T. François, N. Gala, J. Ginestíe and J. C. Ziegler. Simplification of literary and scientific texts to improve reading fluency and comprehension in beginning readers of French. *Applied Psycholinguistics*, 43(2):485–512, 2022.
- [7] E. W. Hermena, D. Drieghe, S. Hellmuth and S. P. Livesedge. Processing of Arabic diacritical marks: phonological–syntactic disambiguation of homographic verbs and visual crowding effects. *JEP: Human Perception and Performance*, 41(2):494–507, 2015.
- [8] B. Alhafni, R. Hazim, J. D. Piñeros Liberato, M. Al Khalil and N. Habash. The SAMER Arabic Text Simplification Corpus. *LREC-COLING*, pp. 16079–16093, 2024.
- [9] K. Elmadani, N. Habash and H. Taha. A Large and Balanced Corpus for Fine-grained Arabic Readability Assessment. *Findings of ACL*, pp. 16376–16400, 2025.
- [10] Baseet: a multi-level dataset for Arabic text simplification. <https://github.com/raya-y/Baseet>
- [11] DAASI: Arabic simplification pairs for government and insurance text. CC0.
- [12] T. Zerrouki and A. Balla. Tashkeela: Novel corpus of Arabic vocalized texts, data for auto-diacritization systems. *Data in Brief*, 11:147–151, 2017.
- [13] C. Scarton and L. Specia. Learning simplifications for specific target audiences. *ACL*, pp. 712–718, 2018.
- [14] L. Martin, É. de la Clergerie, B. Sagot and A. Bordes. Controllable sentence simplification. *LREC*, 2020.
- [15] L. Martin, A. Fan, É. de la Clergerie, A. Bordes and B. Sagot. MUSS: Multilingual unsupervised sentence simplification by mining paraphrases. *LREC*, pp. 1651–1664, 2022.
- [16] W. Xu, C. Napoles, E. Pavlick, Q. Chen and C. Callison-Burch. Optimizing statistical machine translation for text simplification. *TACL*, 4:401–415, 2016.
- [17] F. Alva-Manchego, C. Scarton and L. Specia. The (un)suitability of automatic evaluation metrics for text simplification. *Computational Linguistics*, 47(4):861–889, 2021.
- [18] A. Devaraj, W. Sheffield, B. C. Wallace and J. J. Li. Evaluating factuality in text simplification. *ACL*, pp. 7331–7345, 2022.
- [19] L. Cripwell, J. Legrand and C. Gardent. Simplicity level estimate (SLE): a learned reference-less metric for sentence simplification. *EMNLP*, pp. 12053–12059, 2023.
- [20] D. Heineman, Y. Dou, M. Maddela and W. Xu. Dancing between success and failure: edit-level simplification evaluation using SALSA. *EMNLP*, pp. 3466–3495, 2023.
- [21] M. T. Ribeiro, T. Wu, C. Guestrin and S. Singh. Beyond accuracy: behavioral testing of NLP models with CheckList. *ACL*, pp. 4902–4912, 2020.
- [22] A. M. Bean et al. Measuring what matters: construct validity in large language model benchmarks. *NeurIPS Datasets and Benchmarks*, 2025.
- [23] A. Reuel, A. Hardy, C. Smith, M. Lamparth, M. Hardy and M. J. Kochenderfer. BetterBench: assessing AI benchmarks, uncovering issues, and establishing best practices. *NeurIPS Datasets and Benchmarks*, 2024.

- [24] E. Miller. Adding error bars to evals: a statistical approach to language model evaluations. *arXiv:2411.00640*, 2024.
- [25] D. Li et al. Preference leakage: a contamination problem in LLM-as-a-judge. *ICLR*, 2026.
- [26] E. M. B. Nagoudi, A. Elmadany and M. Abdul-Mageed. AraT5: Text-to-Text Transformers for Arabic Language Generation. *ACL*, pp. 628–647, 2022.
- [27] A. Elmadany, E. M. B. Nagoudi and M. Abdul-Mageed. Octopus: A Multitask Model and Toolkit for Arabic Natural Language Generation. *ArabicNLP*, pp. 232–243, 2023. (Introduces AraT5v2.)
- [28] M. K. Eddine, N. Tomeh, N. Habash, J. Le Roux and M. Vazirgiannis. AraBART: a Pretrained Arabic Sequence-to-Sequence Model for Abstractive Summarization. *WANLP*, pp. 31–42, 2022.
- [29] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li and P. J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *JMLR*, 21(140):1–67, 2020.
- [30] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov and L. Zettlemoyer. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. *ACL*, pp. 7871–7880, 2020.
- [31] D. Samuel, A. Kutuzov, S. Touileb, E. Velldal, L. Øvrelid, E. Rønningstad, E. Sigdel and A. Palatkina. Nor-Bench – A Benchmark for Norwegian Language Models. *NoDaLiDa*, pp. 618–633, 2023.
- [32] L. Burchell et al. An Expanded Massive Multilingual Dataset for High-Performance Language Technologies (HPLT). *ACL*, pp. 17452–17485, 2025.
- [33] O. Obeid et al. CAMEL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing. *LREC*, pp. 7022–7032, 2020.
- [34] abjadai. CATT: Character-based Arabic Tashkeel Transformer. <https://github.com/abjadai/catt>
- [35] T. Zerrouki. Mishkal: Arabic text vocalization. <https://github.com/linuxscout/mishkal>
- [36] Libtashkeel, an ONNX Arabic diacritization model (MIT), via the `text2tashkeel` wrapper.

5. Team Member Review and Comment

<ATTACH A TEAM PICTURE HERE>

NAME	REVIEW and COMMENT
Marwan Elamami	
Abdulrahman Khengari	
Ahmed Alaeb	
Sanad Ali	
Mohammed Thabet	
Abdul Majid Mraied	

6. Instructor Review and Comment

CATEGORY	SCORE	REVIEW and COMMENT
IDEA	__/10	
APPLICATION	__/30	
RESULT	__/30	
PROJECT MANAGEMENT	__/10	
PRESENTATION & REPORT	__/20	
TOTAL	__/100	